Week 9 APP DEV-2

2025 Aug31 ET

1. [Source Q18; MSQ]

Which statements are true about handling background tasks in Flask with Celery?

(a) Using time.sleep() blocks the event loop and is recommended for async tasks

(b) Celery allows long-running tasks to be executed asynchronously

(c) Polling is a strategy to check task status periodically

(d) Flask + Celery + Redis is a common stack for async task management

[Source ans: b, c, d]

2025 Dec18 ET

2. [Source Q7; MCQ]

When comparing Redis with a typical API data store (like a relational database accessed through REST APIs), which of the following statements is true?

(a) Redis can only store strings, unlike API data stores which can handle complex structures.

(b) Redis is an in-memory data structure store, often used as a cache or message broker to complement API-based data stores.

(c) Redis is a relational database and queries data using SQL, just like an API data store.

(d) Redis is slower than API-based databases because it stores everything in memory.

[Source ans: b]

3. [Source Q9; MSQ]

Which of the following describe Redis features correctly?

(a) In-memory database

(b) Can support publish-subscribe (pub/sub)

(c) Can be used as Celery result backend

(d) Requires SQL to query

[Source ans: a, b, c]

4. [Source Q11; MCQ]

What is the purpose of Redis in a Celery-based system?

(a) To automatically retry failed tasks

(b) To execute tasks in parallel

(c) To return computed results in JSON format

(d) To store scheduled tasks and serve as a message broker

[Source ans: d]

2026 Sep13 ET

5. [Source Q2; MCQ]

A news platform wants to send a morning digest email to all subscribers every day at 7 AM. Sweta sets up the beat schedule, starts Redis and Celery Beat, and then claims: "The Worker is optional here. Beat handles the scheduling and the execution of the task, so we do not need a separate Worker process." Is Sweta's claim correct? Choose the best answer.

(a) Yes. For tasks defined in `beat_schedule` , Celery Beat manages both the schedule and the execution, so running the Worker is redundant.

(b) No. Celery Beat cannot trigger tasks that involve database queries or email sending. Those tasks must be triggered manually using `.delay()` from the Flask app.

(c) No. Celery Beat is only a scheduler. At 7 AM, it pushes a task message into the Redis broker and stops there. Without a running Celery Worker to pick up that message, the digest email is never actually sent.

(d) Yes, but only when the task is defined with `.apply_async()` instead of inside `beat_schedule` .In that case, Beat handles execution too.

[Source ans: c]

6. [Source Q5; MCQ]

In a Celery setup, we use two separate Redis databases:

```
celery_app = Celery(
                    'tasks',
                     broker='redis://localhost:6379/1',
                     backend='redis://localhost:6379/2',
                     include=['tasks']
                    )
```

Which of the following best explains the conceptual difference between the broker and the backend?

(a) The backend is only needed when tasks involve sending emails; for tasks that only read from the database, only the broker is required.

(b) The broker and backend must always use different port numbers; using the same Redis port for both would cause a conflict.

(c) The broker distributes tasks evenly across multiple Celery workers, while the backend tracks which worker is currently processing which task.

(d) The broker stores task requests until a Celery worker picks and executes them, while the backend stores the task result after execution.

[Source ans: d]

Shared context (source Q31)

Read the following code carefully and answer the given subquestions. tasks.py:

```
from celery import Celery
app = Celery("myapp", broker="redis://localhost:6379/0",
backend='redis://localhost:6379/0')

@app.task
def add(x, y):
    return x + y
```

Called from a view/script:

```
result = add.delay(4, 6)
print(result)
print(type(result))
```

Checked immediately, right after delay() - worker hasn't processed it yet:

```
from celery.result import AsyncResult
check = AsyncResult(result.id)
print(check.status)
```

Called separately, after waiting for the worker to finish the task:

```
print(check.get(timeout=5))
```

7. [Source Q32; MCQ]

What do the two print statements, print(result) and print(type(result)) in the first code block output?

(a) 10 **and** <class 'int'>

(b) A *task ID string* **and** <class 'celery.result.AsyncResult'>

(c) None **and** <class 'NoneType'>

(d) 10 **and** <class 'celery.result.AsyncResult'>

[Source ans: b]

8. [Source Q33; MCQ]

What do print(check.status) (checked immediately) and the later print(check.get(timeout=5)) (checked after the worker finishes) output?

(a) "PENDING" and 10

(b) "SUCCESS" and 10

(c) 10 and "SUCCESS"

(d) ”PENDING” and ”SUCCESS”

[Source ans: a]

Shiva(25f1000035)